

**LAPORAN PRAKTIKUM KELOMPOK BSD 2**  
**(Aggregation Framework)**



**ANGGOTA KELOMPOK:**

| <b>NAMA</b>                    | <b>NIM</b>       |
|--------------------------------|------------------|
| <b>FAHMI HADI FIRDAUS</b>      | <b>225443007</b> |
| <b>FATIH AKMAL ALQATHANI</b>   | <b>225443009</b> |
| <b>MUHAMMAD IZZUDIN IRSYAD</b> | <b>225443018</b> |
| <b>MUHAMMAD ZAIDAN HABIBI</b>  | <b>225443020</b> |
| <b>NAUFAL RABBANI</b>          | <b>225443022</b> |

**TEKNOLOGI REKAYASA INFORMATIKA INDUSTRI**

**Politeknik Manufaktur Bandung**

**Tahun Ajaran 2025/2026**

## PENDAHULUAN

Pada praktikum ini, kami telah belajar mengenai struktur agregasi database. Dalam pengelolaan basis data modern, kemampuan untuk melakukan agregasi atau penggabungan data secara fleksibel dan efisien menjadi keterampilan yang sangat penting, terutama ketika berhadapan dengan volume data yang besar dan heterogen. MongoDB, sebagai salah satu basis data NoSQL terkemuka, menyediakan framework agregasi yang kuat melalui pipeline aggregation. Framework ini memungkinkan pengguna untuk melakukan transformasi data, pengelompokan, penghitungan statistik, hingga penggabungan antar koleksi dalam satu rangkaian tahapan yang terstruktur.

### A. Tujuan Praktikum

Berdasarkan studi kasus yang diberikan, tujuan pembelajaran dari praktikum ini adalah:

1. Memahami konsep pipeline agregasi di MongoDB dan fungsi masing-masing stage (\$match, \$group, \$project, \$lookup, \$facet, \$bucket, dll.).
2. Mampu merancang dan mengisi data simulasi minimal 200 dokumen untuk produksi harian dan 1000 dokumen untuk inspeksi, sesuai skema yang ditentukan.
3. Menyusun pipeline agregasi untuk menghitung OEE kasar per mesin per bulan, dengan rumus yang melibatkan tiga komponen: efisiensi ketersediaan, efisiensi kinerja, dan efisiensi kualitas.
4. Menerapkan filter untuk menampilkan hanya mesin dengan OEE < 80% pada bulan tertentu.
5. Menggunakan \$bucket untuk mendistribusikan jumlah produksi per shift ke dalam kelompok-kelompok (bucket).
6. Menggabungkan koleksi inspeksi dan target\_kualitas menggunakan \$lookup, serta menambahkan status "OK" / "NOT OK" berdasarkan perbandingan persentase cacat dengan target.
7. Menghitung jumlah batch NOT OK per mesin per minggu menggunakan \$week, lalu menentukan top 3 mesin dengan pelanggaran terbanyak.
8. Menerapkan optimasi agregasi (indeks, \$match awal, proyeksi minimal) untuk meningkatkan performa query.
9. (Opsional) Menggunakan \$facet untuk menghasilkan ringkasan bulanan dalam satu kali query.

## STUDI KASUS

Terdapat sebuah pabrik elektronik ini sedang berfokus pada pengelolaan dan analisis data produksi harian yang memiliki tingkat kompleksitas tinggi akibat banyaknya mesin yang beroperasi secara paralel serta pembagian kerja dalam beberapa shift setiap harinya. Seluruh data produksi dicatat dalam koleksi `produksi_harian` yang memuat atribut seperti identitas mesin, tanggal produksi, shift kerja, target produksi yang ditetapkan, jumlah produk baik (`actual_ok`), jumlah produk cacat (`actual_reject`), serta durasi operasi dan durasi waktu tersedia dalam satuan menit.

Permasalahan utama yang dihadapi adalah bagaimana mengolah data mentah tersebut menjadi informasi yang dapat difilter, diatur untuk mengevaluasi performa produksi secara menyeluruh, baik dari sisi kuantitas maupun kualitas.

### Pipeline data dummy:

```
from pymongo import MongoClient
from datetime import datetime, timedelta
import random

client = MongoClient('mongodb://localhost:27017')
db = client['studi_kasus_oe']
collection = db['produksi_harian']
collection.drop() # Mulai bersih

mesin_list = [f"M{i:02d}" for i in range(1, 11)]
start_date = datetime(2026, 4, 1)
docs = []

for i in range(1000):
    # Acak hari dalam 90 hari
    day_offset = random.randint(0, 90)
    tanggal = start_date + timedelta(days=day_offset)
    shift = random.randint(1, 3)
    mesin = random.choice(mesin_list)
    target = random.randint(200, 500)
    actual_ok = int(target * random.uniform(0.6, 1.0)) # Kadang di bawah target
    actual_reject = int(actual_ok * random.uniform(0, 0.15)) # reject sampai 15%
    durasi_tersedia = 480 # 8 jam
    durasi_operasi = int(durasi_tersedia * random.uniform(0.7, 1.0))

    doc = {
        "mesin": mesin,
        "tanggal": tanggal,
        "shift": shift,
        "target": target,
        "actual_ok": actual_ok,
        "actual_reject": actual_reject,
        "durasi_operasi": durasi_operasi,
        "durasi_tersedia": durasi_tersedia
    }
    docs.append(doc)

if len(docs) >= 500:
    collection.insert_many(docs)
    docs.clear()

if docs:
```

```
collection.insert_many(docs)
print("Data berhasil dibuat. Total:", collection.count_documents({}))
```

## Output:

```
[Running] python -u "c:\Users\ASUS\OneDrive\Dokumen\.vscode\mongodb\Agregasi_Framework\tugas5.py"
Data inspeksi selesai. Total: 1500
Data target kualitas selesai. Total: 2000

[Done] exited with code=0 in 1.429 seconds
```

## Manajemen ingin:

### 1. Menghitung OEE secara kasar

```
db.produksi_harian.aggregate([
{
  $project: {
    oee: {
      $multiply: [
        { $divide: ["$actual_ok", "$target"] },
        { $divide: ["$durasi_operasi_menit", "$durasi_tersedia_menit"] },
        { $divide: ["$actual_ok", { $add: ["$actual_ok", "$actual_reject"] } ] }
      ]
    }
  }
}]
```

## Output:

```
{
  "_id": ObjectId('69f81013a8384375cdd4b163'),
  "oee": 0.6456672872581963
}
{
  "_id": ObjectId('69f81013a8384375cdd4b164'),
  "oee": 0.4639353202348863
}
{
  "_id": ObjectId('69f81013a8384375cdd4b165'),
  "oee": 0.4642383438262218
}
{
  "_id": ObjectId('69f81013a8384375cdd4b166'),
  "oee": 0.6023618781705122
}
{
  "_id": ObjectId('69f81013a8384375cdd4b167'),
  "oee": 0.7345164968524702
}
{
  "_id": ObjectId('69f81013a8384375cdd4b168'),
  "oee": 0.7491267108621547
}
{
  "_id": ObjectId('69f81013a8384375cdd4b169'),
  "oee": 0.7632239854361869
}
{
  "_id": ObjectId('69f81013a8384375cdd4b16a'),
  "oee": 0.6473982388482654
}
```

## Penjelasan:

Query ini digunakan untuk menghitung nilai OEE pada setiap produksi dengan memanfaatkan \$project. Perhitungan OEE dilakukan dengan

mengalikan tiga komponen utama, yaitu rasio actual\_ok dibagi target, rasio durasi\_operasi\_menit dibagi durasi\_tersedia\_menit. Hasil ini untuk mencari nilai yang dapat mempresentasikan OEE.

## 2. Menghitung OEE mesin perbulan

```
db.produksi_harian.aggregate([
  {
    $group: {
      _id: {
        mesin: "$mesin",
        bulan: { $month: "$tanggal" },
        tahun: { $year: "$tanggal" }
      },
      total_target: { $sum: "$target" },
      total_ok: { $sum: "$actual_ok" },
      total_reject: { $sum: "$actual_reject" },
      total_operasi: { $sum: "$durasi_operasi_menit" },
      total_tersedia: { $sum: "$durasi_tersedia_menit" }
    },
  },
  {
    $project: {
      _id: 0,
      mesin: "$_id.mesin",
      bulan: "$_id.bulan",
      tahun: "$_id.tahun",

      availability: {
        $divide: ["$total_operasi", "$total_tersedia"]
      },
      performance: {
        $divide: ["$total_ok", "$total_target"]
      },
      quality: {
        $divide: [
          "$total_ok",
          { $add: ["$total_ok", "$total_reject"] }
        ]
      },
      OEE: {
        $multiply: [
          { $divide: ["$total_ok", "$total_target"] },
          { $divide: ["$total_operasi", "$total_tersedia"] },
          {
            $divide: [
              "$total_ok",
              { $add: ["$total_ok", "$total_reject"] }
            ]
          }
        ]
      }
    }
  },
  {
    $sort: { OEE: 1 }
  }
])
```

**Output:**

```

mesin: 'M07',
bulan: 3,
tahun: 2026,
availability: 0.8885416666666667,
performance: 0.6530849825378346,
quality: 0.9428571428571428,
OEE: 0.5471336063529021
}
{
  mesin: 'M03',
  bulan: 4,
  tahun: 2026,
  availability: 0.8320833333333333,
  performance: 0.7640713865463816,
  quality: 0.9148761300927557,
  OEE: 0.5816517726902519
}
{
  mesin: 'M06',
  bulan: 3,
  tahun: 2026,
  availability: 0.7666666666666667,
  performance: 0.8484848484848485,
  quality: 0.9090909090909091,
  OEE: 0.5913682277318641
}
{
  mesin: 'M02',
  bulan: 5,
  tahun: 2026,
  availability: 0.8234722222222223,
  performance: 0.7799250072108451,
  quality: 0.9266620973269363,
  OEE: 0.5951455617624507
}
{
  mesin: 'M10',
  bulan: 4,
  tahun: 2026,
  availability: 0.8476641414141414,
  performance: 0.7660137201421605,
  quality: 0.9246732515215006,
  OEE: 0.6004110201221067
}

```

## Penjelasan:

Query tersebut bertujuan untuk menganalisis kinerja produk dalam skala bulanan berdasarkan mesin, dengan dikelompokkan datanya melalui \$group berdasarkan mesin, bulan, dan tahun. \$project disini berfungsi untuk mengolah beberapa metrik utama yaitu untuk ketersediaan mesin, performance, quality, yang kemudian akan dihitung untuk menghasilkan nilai OEE.

### 3. Menemukan mesin dengan OEE di bawah 80% pada bulan tertentu

```

db.produksi_harian.aggregate([
  {
    $match: {
      tanggal: {
        $gte: ISODate("2026-06-01"),
        $lt: ISODate("2026-07-01")
      }
    }
  }
])

```

```

    }
  },
  {
    $group: {
      _id: "$mesin",
      total_target: { $sum: "$target" },
      total_ok: { $sum: "$actual_ok" },
      total_reject: { $sum: "$actual_reject" },
      total_operasi: { $sum: "$durasi_operasi_menit" },
      total_tersedia: { $sum: "$durasi_tersedia_menit" }
    }
  },
  {
    $project: {
      _id: 0,
      mesin: "$_id",
      OEE: {
        $multiply: [
          { $divide: ["$total_ok", "$total_target"] },
          { $divide: ["$total_operasi", "$total_tersedia"] },
          {
            $divide: [
              "$total_ok",
              { $add: ["$total_ok", "$total_reject"] }
            ]
          }
        ]
      }
    }
  },
  {
    $match: {
      OEE: { $lt: 0.8 }
    }
  },
  {
    $sort: { OEE: 1 }
  }
}
1)

```

**Output:**

```

{
  mesin: 'M01',
  OEE: 0.6252891106004828
}
{
  mesin: 'M05',
  OEE: 0.6327890743182639
}
{
  mesin: 'M06',
  OEE: 0.6344009303535765
}
{
  mesin: 'M02',
  OEE: 0.6351945864443898
}
{
  mesin: 'M09',
  OEE: 0.6427228405160971
}
{
  mesin: 'M03',
  OEE: 0.6502799510761668
}
{
  mesin: 'M07',
  OEE: 0.6531342010404729
}
{
  mesin: 'M08',
  OEE: 0.654323232075098
}
{
  mesin: 'M10',
  OEE: 0.6616641961857015
}
{
  mesin: 'M04',
  OEE: 0.6669202398355659
}
}

```

### Penjelasan:

Query ini bertujuan untuk mengidentifikasi mesin dengan berdasarkan kinerja mesin pada periode Juni 2026. \$match berguna untuk memfilter data apa yang mau ditampilkan, dalam kasus ini aktivitas mesin pada bulan Juni 2026. Selanjutnya, \$group untuk menghitung total target, hasil

cacat, serta durasi operasi. Terakhir, \$project untuk menghitung nilai OEE dari masing - masing mesin.

#### 4. Distribusi jumlah produksi per shift dalam bentuk bucket

```
db.produksi_harian.aggregate([
  {
    $bucket: {
      groupBy: "$actual_ok",
      boundaries: [0, 200, 300, 400, 600],
      default: "di atas 600",
      output: {
        jumlah_data: { $sum: 1 },
        rata_reject: { $avg: "$actual_reject" }
      }
    }
  }
])
```

#### Output:

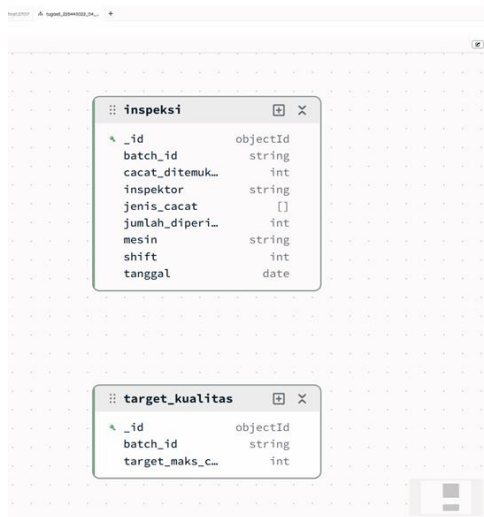
```
< {
  _id: 0,
  jumlah_data: 173,
  rata_reject: 12.22543352601156
}
{
  _id: 200,
  jumlah_data: 423,
  rata_reject: 18.79432624113475
}
{
  _id: 300,
  jumlah_data: 303,
  rata_reject: 26.392739273927393
}
{
  _id: 400,
  jumlah_data: 101,
  rata_reject: 30.257425742574256
}
```

#### Penjelasan:

Query ini berfungsi untuk mengelompokkan produksi berdasarkan jumlah hasil actual\_ok ke dalam beberapa rentang nilai menggunakan tahap \$bucket. Setiap rentang (0 - 200, 200 - 300, 300 - 400, 400 - 600, dan di atas 600) akan menampilkan jumlah data yang masuk ke dalam kategori tersebut.

# TUGAS TERSTRUKTUR

## Diagram Koleksi



## Contoh Koleksi

### a. Inspeksi

```
_id: ObjectId('69f81f174db2247d6078aa65'),
batch_id: 'B1364',
mesin: 'M04',
tanggal: 2026-05-25T15:00:00.000Z,
shift: 2,
inspektor: 'Dian',
jumlah_diperiksa: 213,
cacat_ditemukan: 18,
jenis_cacat: [
  'pori',
  'dimensi'
]
```

### b. Target\_Kualitas

```
_id: ObjectId('69f81f174db2247d6078b041'),
batch_id: 'B0001',
target_maks_cacat: 10
```

## 1.1. Pipe Line 1

### a. Syntax

```
db.inspeksi.aggregate([
  db.inspeksi.aggregate([
    { $addField: {
      persen_cacat: {
        $multiply: [
          { $divide: ["$cacat_ditemukan", "$jumlah_diperiksa"] },
          100
        ]
      }
    }
  ]},
  { $sort: { persen_cacat: -1 } },
  { $limit: 10 }
])
```

### b. Hasil

```
{
  "_id": ObjectId("69f81f174db2247d6078aedc"),
  "batch_id": "B0770",
  "mesin": "M05",
  "tanggal": 2026-05-05T22:11:00.000Z,
  "shifts": 3,
  "inspektors": "Eka",
  "jumlah_diperiksa": 341,
  "cacat_ditemukan": 34,
  "jenis_cacat": [
    'gores'
  ],
  "persen_cacat": 9.970674460803510
}
{
  "_id": ObjectId("69f81f174db2247d6078adea"),
  "batch_id": "B0392",
  "mesin": "M08",
  "tanggal": 2026-04-16T22:11:00.000Z,
  "shifts": 3,
  "inspektors": "Budl",
  "jumlah_diperiksa": 273,
  "cacat_ditemukan": 27,
  "jenis_cacat": [
    'dimensi',
    'gores',
    'retak'
  ],
  "persen_cacat": 9.8910909010989
}
{
  "_id": ObjectId("69f81f174db2247d6078aef1"),
  "batch_id": "B0862",
  "mesin": "M02",
  "tanggal": 2026-06-06T04:29:00.000Z,
  "shifts": 1,
  "inspektors": "Andi",
  "jumlah_diperiksa": 487,
  "cacat_ditemukan": 48,
  "jenis_cacat": [
    'pori'
  ],
  "persen_cacat": 9.85626283073565
}
{
  "_id": ObjectId("69f81f174db2247d6078acf2"),
  "batch_id": "B0612",
  "mesin": "M09",
  "tanggal": 2026-06-03T21:43:00.000Z,
  "shifts": 2,
  "inspektors": "Andi",
  "jumlah_diperiksa": 400,
  "cacat_ditemukan": 40,
  "jenis_cacat": [
    'retak',
    'dimensi'
  ],
  "persen_cacat": 9.852216748768473
}
{
  "_id": ObjectId("69f81f174db2247d6078abf0"),
  "batch_id": "B1780",
  "mesin": "M07",
  "tanggal": 2026-05-03T07:16:00.000Z,
  "shifts": 1,
  "inspektors": "Andi",
  "jumlah_diperiksa": 122,
  "cacat_ditemukan": 12,
  "jenis_cacat": [
    'retak',
    'bengkok',
    'warna'
  ],
  "persen_cacat": 9.830065573770492
}
{
  "_id": ObjectId("69f81f174db2247d6078ad02"),
  "batch_id": "B1767",
  "mesin": "M03",
  "tanggal": 2026-04-16T02:40:00.000Z,
  "shifts": 1,
  "inspektors": "Eka",
  "jumlah_diperiksa": 316,
  "cacat_ditemukan": 31,
  "jenis_cacat": [
    'warna',
    'bengkok'
  ],
  "persen_cacat": 9.81812658227848
}
{
  "_id": ObjectId("69f81f174db2247d6078add2"),
  "batch_id": "B1711",
  "mesin": "M03",
  "tanggal": 2026-04-07T18:44:00.000Z,
  "shifts": 1,
  "inspektors": "Andi",
  "jumlah_diperiksa": 400,
  "cacat_ditemukan": 40,
  "jenis_cacat": [
    'warna',
    'gores',
    'retak'
  ],
  "persen_cacat": 9.803921568027452
}
{
  "_id": ObjectId("69f81f174db2247d6078adc9"),
  "batch_id": "B0200",
  "mesin": "M02",
  "tanggal": 2026-04-16T03:42:00.000Z,
  "shifts": 1,
  "inspektors": "Diana",
  "jumlah_diperiksa": 327,
  "cacat_ditemukan": 32,
  "jenis_cacat": [
    'pori',
    'dimensi'
  ],
  "persen_cacat": 9.785932721712538
}
{
  "_id": ObjectId("69f81f174db2247d6078ae79"),
  "batch_id": "B0958",
  "mesin": "M05",
  "tanggal": 2026-06-24T06:23:00.000Z,
  "shifts": 1,
  "inspektors": "Andi",
  "jumlah_diperiksa": 143,
  "cacat_ditemukan": 14,
  "jenis_cacat": [
    'dimensi'
  ],
  "persen_cacat": 9.79020979020979
}
{
  "_id": ObjectId("69f81f174db2247d6078ac7f"),
  "batch_id": "B0388",
  "mesin": "M07",
  "tanggal": 2026-04-26T23:08:00.000Z,
  "shifts": 3,
  "inspektors": "Dian",
  "jumlah_diperiksa": 327,
  "cacat_ditemukan": 32,
  "jenis_cacat": [
    'pori',
    'dimensi'
  ],
  "persen_cacat": 9.785932721712538
}
```

### c. Penjelasan

Pada pipeline pertama adalah untuk menampilkan batch\_id, mesin, jumlah\_diperiksa, cacat\_ditemukan dan persen cacat dengan rumus ( $\text{cacat\_ditemukan} / \text{jumlah\_diperiksa} * 100$ ). Lalu urutkan berdasarkan persentase tertinggi.

## 1.1. Pipe Line 2

### a. Syntax

```
db.inspeksi.aggregate([
  { $lookup: {
    from: "target_kualitas",
    localField: "batch_id",
    foreignField: "batch_id",
    as: "target_info"
  }},
  { $unwind: "$target_info" },
  { $addFields: {
    persen_cacat: { $multiply: [ { $divide: ["$cacat_ditemukan",
"$jumlah_diperiksa" ] }, 100 ] },
    target_maks: "$target_info.target_maks_cacat"
  }},
  { $addFields: {
    status: { $cond: { if: { $gt: ["$persen_cacat", "$target_maks"] }, then: "NOT
OK", else: "OK" } }
  }},
  { $project: {
    _id: 0,
    batch_id: 1,
    mesin: 1,
    persen_cacat: 1,
    target_maks: 1,
    status: 1,
    selisih: { $subtract: ["$persen_cacat", "$target_maks"] }
  }},
  { $sort: { selisih: -1 } },
  { $limit: (10)}
])
```

## b. Hasil

```
{
  batch_id: 'B1291',
  mesin: 'M04',
  persen_cacat: 9.574468885106384,
  target_maks: 1,
  status: 'NOT OK',
  selisih: 8.574468885106384
}
{
  batch_id: 'B0787',
  mesin: 'M04',
  persen_cacat: 9.523889523889524,
  target_maks: 1,
  status: 'NOT OK',
  selisih: 8.523889523889524
}
{
  batch_id: 'B1334',
  mesin: 'M10',
  persen_cacat: 9.482758620689655,
  target_maks: 1,
  status: 'NOT OK',
  selisih: 8.482758620689655
}
{
  batch_id: 'B1334',
  mesin: 'M05',
  persen_cacat: 9.45945945945946,
  target_maks: 1,
  status: 'NOT OK',
  selisih: 8.45945945945946
}
{
  batch_id: 'B1147',
  mesin: 'M06',
  persen_cacat: 9.45945945945946,
  target_maks: 1,
  status: 'NOT OK',
  selisih: 8.45945945945946
}
{
  batch_id: 'B0640',
  mesin: 'M05',
  persen_cacat: 9.352517985611511,
  target_maks: 1,
  status: 'NOT OK',
  selisih: 8.352517985611511
}
}

< {
  batch_id: 'B0392',
  mesin: 'M08',
  persen_cacat: 9.89010989010989,
  target_maks: 1,
  status: 'NOT OK',
  selisih: 8.89010989010989
}
{
  batch_id: 'B0388',
  mesin: 'M07',
  persen_cacat: 9.785932721712538,
  target_maks: 1,
  status: 'NOT OK',
  selisih: 8.785932721712538
}
{
  batch_id: 'B1644',
  mesin: 'M10',
  persen_cacat: 9.765625,
  target_maks: 1,
  status: 'NOT OK',
  selisih: 8.765625
}
{
  batch_id: 'B0118',
  mesin: 'M01',
  persen_cacat: 9.670329670329672,
  target_maks: 1,
  status: 'NOT OK',
  selisih: 8.670329670329672
}
}
```

## c. Penjelasan

Gabungkan koleksi inspeksi dan target\_kualitas berdasarkan batch\_id, lalu akan dihitung persentase cacat dan bandingkan dengan target\_maks\_cacat. Jika persen\_cacat > target\_maks\_cacat -> status not ok, sebaliknya ok. Menampilkan selisihnya juga

## 1.2. Pipe Line 3

### a. Syntax

```
db.inspeksi.aggregate([
  { $lookup: {
    from: "target_kualitas",
    localField: "batch_id",
    foreignField: "batch_id",
    as: "target_info"
  }},
  { $unwind: "$target_info" },
  { $addFields: {
    persen_cacat: { $multiply: [ { $divide: ["$cacat_ditemukan", "$jumlah_diperiksa" ] },
    100 ] }
  }},
  { $addFields: {
    status: { $cond: { if: { $gt: ["$persen_cacat", "$target_info.target_maks_cacat" ] },
    then: "NOT OK", else: "OK" } }
  }},
  { $match: { status: "NOT OK" } },
  { $group: {
    _id: {
      mesin: "$mesin",
      minggu: { $week: "$tanggal" },
      tahun: { $year: "$tanggal" }
    },
    jumlah_NOT_OK: { $sum: 1 }
  }},
  { $sort: { "_id.tahun": 1, "_id.minggu": 1, "_id.mesin": 1 } }
])
```

**b. Hasil**

```
< {
  _id: {
    mesin: 'M01',
    minggu: 13,
    tahun: 2026
  },
  jumlah_NOT_OK: 1
},
{
  _id: {
    mesin: 'M02',
    minggu: 13,
    tahun: 2026
  },
  jumlah_NOT_OK: 2
},
{
  _id: {
    mesin: 'M04',
    minggu: 13,
    tahun: 2026
  },
  jumlah_NOT_OK: 2
},
{
  _id: {
    mesin: 'M05',
    minggu: 13,
    tahun: 2026
  },
  jumlah_NOT_OK: 6
},
{
  _id: {
    mesin: 'M08',
    minggu: 13,
    tahun: 2026
  },
  jumlah_NOT_OK: 3
},
{
  _id: {
    mesin: 'M09',
    minggu: 13,
    tahun: 2026
  },
  jumlah_NOT_OK: 5
},
{
  _id: {
    mesin: 'M10',
    minggu: 13,
    tahun: 2026
  },
  jumlah_NOT_OK: 2
},
{
  _id: {
    mesin: 'M01',
    minggu: 14,
    tahun: 2026
  },
  jumlah_NOT_OK: 4
},
{
  _id: {
    mesin: 'M02',
    minggu: 14,
    tahun: 2026
  },
  jumlah_NOT_OK: 3
},
{
  _id: {
    mesin: 'M03',
    minggu: 14,
    tahun: 2026
  },
  jumlah_NOT_OK: 2
}
```

**c. Penjelasan**

Mengekstrak minggu dari tanggal menggunakan \$week, lalu filter hanya batch dengan status NOT OK, kemudian grup per mesin dan minggu. Menggunakan operasi \$match di awal hanya batch NOT OK, tapi kita harus kalkulasi lalu memfilter menggunakan \$match.

### 1.3. Pipe Line 4

#### a. Syntax

```
db.inspeksi.aggregate([
  { $match: {
    tanggal: { $gte: ISODate("2026-05-01"), $lt: ISODate("2026-06-01") }
  }},
  { $lookup: {
    from: "target_kualitas",
    localField: "batch_id",
    foreignField: "batch_id",
    as: "target_info"
  }},
  { $unwind: "$target_info" },
  { $addFields: {
    persen_cacat: { $multiply: [ { $divide: ["$cacat_ditemukan", "$jumlah_diperiksa"] },
    100 ] }
  }},
  { $addFields: {
    status: { $cond: { if: { $gt: ["$persen_cacat", "$target_info.target_maks_cacat"] },
    then: "NOT OK", else: "OK" } }
  }},
  { $match: { status: "NOT OK" } },
  { $group: {
    _id: "$mesin",
    total_NOT_OK: { $sum: 1 }
  }},
  { $sort: { total_NOT_OK: -1 } },
  { $limit: 3 }
])
```

**b. Hasil**

```
{
  _id: 'M04',
  total_NOT_OK: 22
}
{
  _id: 'M07',
  total_NOT_OK: 21
}
{
  _id: 'M05',
  total_NOT_OK: 17
}
{
  _id: 'M03',
  total_NOT_OK: 14
}
{
  _id: 'M06',
  total_NOT_OK: 13
}
{
  _id: 'M09',
  total_NOT_OK: 11
}
{
  _id: 'M08',
  total_NOT_OK: 11
}
{
  _id: 'M01',
  total_NOT_OK: 10
}
{
  _id: 'M02',
  total_NOT_OK: 10
}
{
  _id: 'M10',
  total_NOT_OK: 9
}
```

**c. Penjelasan**

Mencari top 3 mesin dengan jumlah NOT OK terbanyak dalam satu bulan tertentu menggunakan operasi \$match untuk memfilter bulan lalu lakukan join dan penentuan status, lalu grup per mesin dan hitung jumlah NOT OK, lalu urutkan.

## 1.4. Pipe Line 5

### a. Syntax

```
db.inspeksi.aggregate([
  { $match: { tanggal: { $gte: ISODate("2026-06-01"), $lt: ISODate("2026-07-01") } } },
  { $lookup: {
    from: "target_kualitas",
    localField: "batch_id",
    foreignField: "batch_id",
    as: "target_info"
  }},
  { $unwind: "$target_info" },
  { $addFields: {
    persen_cacat: { $multiply: [ { $divide: ["$cacat_ditemukan", "$jumlah_diperiksa"] },
    100 ] },
    status: { $cond: { if: { $gt: [ { $multiply: [ { $divide: ["$cacat_ditemukan",
"$jumlah_diperiksa"] }, 100 ] }, "$target_info.target_maks_cacat" ] }, then: "NOT OK", else:
"OK" } }
  }},
  { $facet: {
    "total_batch": [
      { $count: "count" }
    ],
    "rata_cacat": [
      { $group: { _id: null, avg_cacat: { $avg: "$persen_cacat" } } }
    ],
    "mesin_terburuk": [
      { $match: { status: "NOT OK" } },
      { $group: { _id: "$mesin", jumlah: { $sum: 1 } } },
      { $sort: { jumlah: -1 } },
      { $limit: 3 }
    ]
  }
}]
```

**b. Hasil**

```
{
  mesin_terburuk: [
    {
      _id: 'M06',
      jumlah: 15
    },
    {
      _id: 'M09',
      jumlah: 13
    },
    {
      _id: 'M08',
      jumlah: 12
    },
    {
      _id: 'M02',
      jumlah: 12
    },
    {
      _id: 'M07',
      jumlah: 10
    }
  ],
  mesin_terbaik: [
    {
      _id: 'M04',
      jumlah: 20
    },
    {
      _id: 'M01',
      jumlah: 19
    },
    {
      _id: 'M03',
      jumlah: 19
    },
    {
      _id: 'M10',
      jumlah: 17
    },
    {
      _id: 'M05',
      jumlah: 15
    }
  ],
}
```

```
{
  total_batch: [
    {
      count: 514
    }
  ],
  rata_cacat: [
    {
      _id: null,
      avg_cacat: 4.9583708161932725
    }
  ],
}
```

**c. Penjelasan**

Menggunakan operasi \$facet untuk memperoleh tiga informasi dalam satu query:

1. Total produksi (jumlah batch) per bulan.
2. Rata-rata persentase cacat per bulan.
3. Mesin dengan jumlah NOT OK terbanyak per bulan.

## **KESIMPULAN**

Berdasarkan praktikum yang telah dilakukan, program yang kami eksekusi adalah Aggregation Framework MongoDB dan berhasil dijalankan dengan baik untuk menganalisis data pabrik. Pada tahap awal kami berhasil mengeksekusi data dummy, kemudian kami mengolah data tersebut untuk mencari nilai efektivitas produksi atau Overall Equipment Effectiveness (OEE). Melalui perhitungan ini, sistem dapat secara akurat menggabungkan nilai kinerja, ketersediaan mesin, dan kualitas produk untuk menilai performa tiap mesin. Hasilnya, kami bisa dengan mudah menyaring mesin-mesin yang kinerjanya kurang maksimal, yaitu di bawah batas 80% serta melihat data jumlah produksi berdasarkan shift kerja karyawan.

Selain itu, sistem ini juga sangat berguna untuk membangun dashboard pemantauan kualitas pabrik. Sistem mampu menghitung persentase cacat pada setiap batch produksi dan membandingkannya dengan target maksimal secara otomatis, lalu memberikan status "OK" atau "NOT OK" pada batch tersebut. Selain itu sistem dapat melacak riwayat dari waktu ke waktu untuk menemukan tiga mesin terburuk yang paling sering menghasilkan produk cacat. Secara keseluruhan, penerapan metode agregasi ini terbukti sangat ampuh dan efisien dalam mengubah data mentah yang rumit menjadi informasi yang sederhana dan langsung bisa digunakan oleh manajemen untuk mengambil keputusan perbaikan mesin.